

UNIVERSIDAD NACIONAL DE CÓRDOBA FACULTAD DE
CIENCIAS EXACTAS, FÍSICAS Y NATURALES



Arquitectura de Computadoras

Trabajo Práctico 1 - ALU

Alumnos:
Santiago Madrid
Leandro Malano

2025

Introducción

En este trabajo se busca implementar una ALU en una FPGA. La misma debe ser parametrizable para su posterior implementación.

El desarrollo de la misma debe ser testeado mediante TestBench que genera entradas aleatorias y chequea automáticamente la salida. Además se generan test específicos para comprobar situaciones límites.

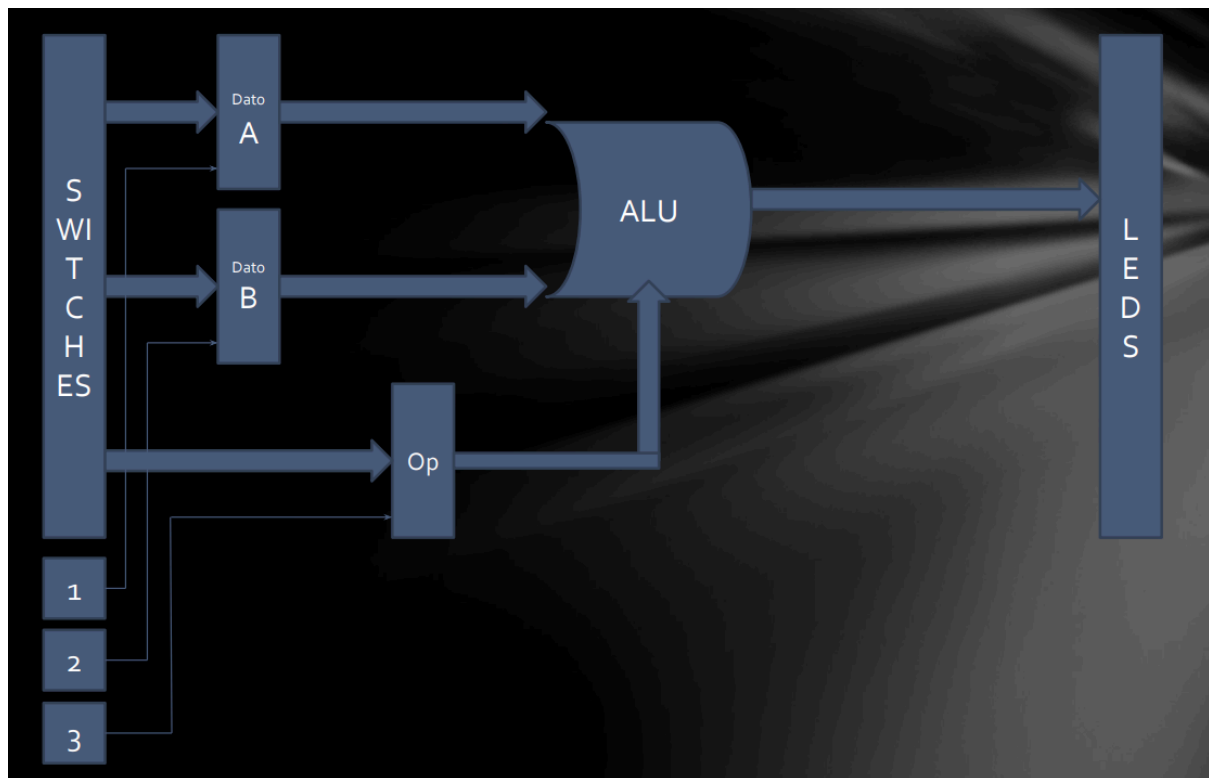
Las operaciones que debe poder realizar la ALU son las siguientes:

Operación	Código
ADD	100000
SUB	100010
AND	100100
OR	100101
XOR	100110
SRA	000011
SRL	000010
NOR	100111

Desarrollo

Como las placas FPGA típicamente no disponen de grandes cantidades de interruptores, será necesario contar con botones que guarden el estado actual de los Switches en determinados registros, para poder almacenar los valores con los que luego se va a operar.

La siguiente imagen muestra el diseño con las anteriores consideraciones



El sistema se compone del módulo ALU, que es responsable de realizar las operaciones definidas según las entradas proporcionadas. Inicialmente, creamos el módulo siguiendo las especificaciones del enunciado. Los parámetros principales son la cantidad de bits de los operandos y del código de operación. De manera predeterminada, el módulo define 8 bits para los operandos y 6 bits para el código de operación. Las entradas del módulo son:

i_data_a: Representa el operando A.

i_data_b: Representa el operando B.

i_data_op: Representa el código de operación OP. 2

La lógica de las operaciones se implementa dentro de un bloque always. Es importante señalar que no se puede asignar directamente el valor del resultado a la salida , ya que esta es de tipo wire, que representa una señal de conexión continua.

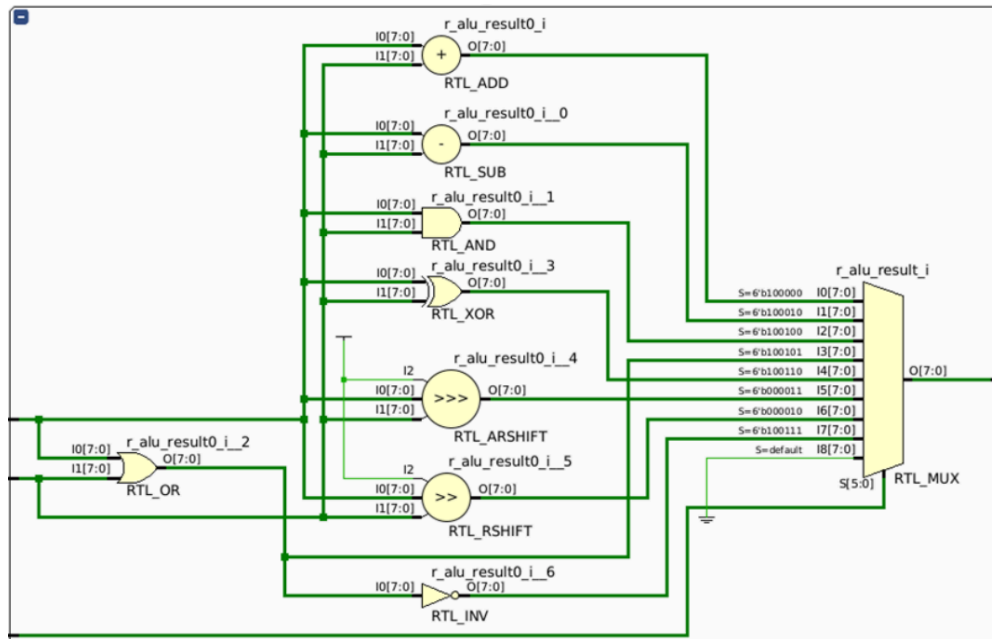
Debido a que tratamos el módulo como puramente combinacional, el resultado de las operaciones se calcula en momentos específicos, en función de las variaciones en las entradas.

Para solucionar este inconveniente, utilizamos un registro auxiliar al que modificamos dentro del bloque always como si fuera nuestro resultado. Fuera del bloque always, asignamos el valor de este registro temporal a la señal de salida.

El bloque always se configura como always @(*), lo que significa que es de tipo combinacional y se ejecuta cada vez que alguna de las señales que utilizamos en su interior varía. Los resultados se asignan según la operación indicada por el código de operación. De manera descriptiva, cada operación se representa con un case, y se puede mejorar la legibilidad utilizando localparam para asignar etiquetas más amigables a las operaciones.

Finalmente, la salida del módulo se asigna mediante un assign al valor del registro temporal, garantizando que el resultado correcto se propague hacia la salida del módulo.

Desde el punto de vista del circuito físico, este módulo se representa a través del diagrama que se muestra en la figura 2, en la cual se aprecia que el componente principal es un multiplexor que en función de su selector elige una de las 8 entradas correspondientes a cada una de las operaciones disponibles.



Una vez definido el módulo, podemos simular su comportamiento mediante los testbenches. Para ello, volvemos a definir un módulo que no tiene entradas ni salidas directas, sino que se mapea sobre el módulo que habíamos definido previamente. En este contexto, volvemos a declarar los parameters, utilizamos localparam para definir operaciones, y empleamos registros signados que representan las entradas y salidas, así como registros auxiliares.

Luego, realizamos una instancia del módulo de la ALU, utilizando nuestros registros como referencia. Y finalmente, en un bloque initial, además de hacer las asignaciones necesarias, llevamos a cabo cálculos y verificamos los resultados. Para esto, utilizamos un arreglo de operaciones y lo recorremos, aplicando cada operación sobre los operandos correspondientes.

El sistema también incluye un módulo superior (TOP), que instancia y utiliza submódulos, como el módulo ALU. La idea es emplear los switches físicos de la placa para cargar los valores de los operandos A, B y el código de operación.

Los parámetros del módulo TOP son similares a los del módulo ALU, estableciendo la cantidad de bits para los operandos y el código de operación. Además, se añaden dos parámetros adicionales: uno para definir

la cantidad de bits de los switches (8 bits) y otro para los botones (3 bits).

Entradas y salidas del módulo:

- Entradas: Están definidas por los botones A, B y el botón de operación, representados en el array `i_button`, así como por el array `i_switch`, que contiene la palabra de bits ingresada.
- Salidas: Consisten en el resultado de enviar los operandos y el código de operación al módulo ALU. Este resultado se visualizará, por ejemplo, en los LEDs de la placa.

Parámetros locales: Se utilizan para simplificar la lectura del código, mejorando su claridad.

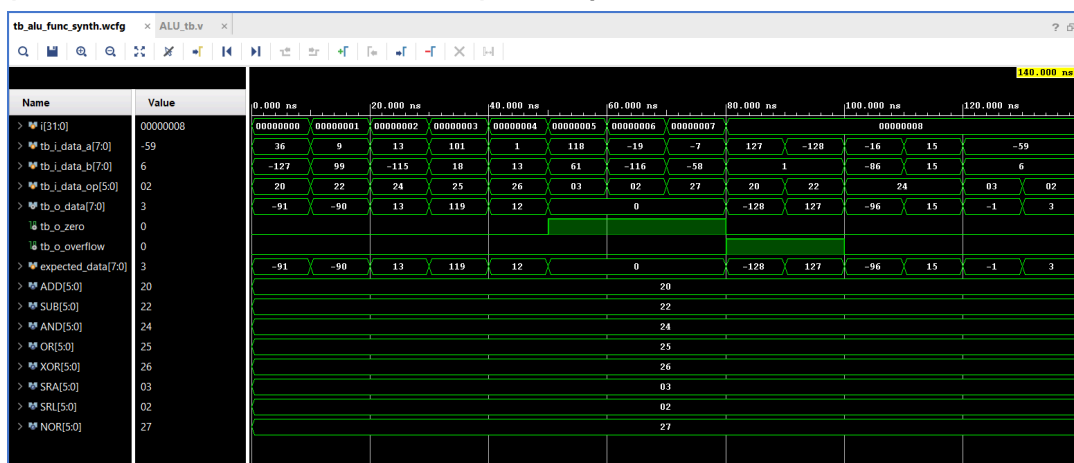
Registros internos: Dado que las entradas son del tipo wire, se requieren registros internos para almacenar el estado de los cambios de las entradas.

Instanciación del módulo ALU: En este paso, los registros se mapean a los wires del módulo, permitiendo la conexión y el envío de información entre los módulos.

Bloque always secuencial: Este bloque se activa con el flanco positivo del reloj (posedge `i_clk`). Las operaciones son síncronas con el clock, capturando los datos de entrada en cada flanco positivo y actualizando los registros de acuerdo con el botón presionado. Se utiliza la asignación no bloqueante (`<=`), lo que permite que todas las operaciones dentro del bloque always se ejecuten simultáneamente.

Análisis temporal de las señales

Una vez descritos los módulos de trabajo, haremos un sencillo análisis de tiempo de las señales involucradas en el sistema. La herramienta de testbench nos permite ingresar un retardo temporal ante cada una de las instrucciones ejecutadas, de forma de poder simular una actuación real de un operario accionando los interruptores y botones.



Conclusión

A lo largo de este trabajo práctico, aplicamos y afianzamos conceptos fundamentales del diseño digital utilizando Verilog, un lenguaje de descripción de hardware (HDL). Implementamos con éxito una Unidad Aritmético-Lógica (ALU) en una FPGA, cumpliendo con las especificaciones del enunciado y garantizando su correcto funcionamiento mediante testbenches que incluyeron la generación de entradas aleatorias y su respectiva verificación.

Los resultados obtenidos fueron correctos para todas las operaciones aritméticas y lógicas, aunque, para aquellas operaciones que producen un resultado no representable con la cantidad de bits definida, se detecta un overflow, generando un resultado que corresponde al módulo de dicho valor. En nuestro caso, la ALU trabajó con 8 bits, pero al ser parametrizable, este valor puede ajustarse fácilmente en futuras implementaciones.

Durante el desarrollo, comprendimos las diferencias entre los bloques secuenciales y combinacionales en Verilog, utilizando el bloque always. El uso de registros temporales nos permitió gestionar las señales de manera eficiente, asegurando la correcta asignación de resultados en bloques combinacionales. También decidimos modificar la lógica de las entradas controladas por botones, optando por utilizar el flanco de subida del reloj (posedge i_clk), lo que simplificó el manejo de señales y mejoró la sincronización general del diseño.

Finalmente, la simulación y validación del diseño mediante las herramientas de simulación de Vivado nos permitió analizar el comportamiento temporal del sistema y asegurar que la ALU funcionara correctamente bajo diversas condiciones de prueba.

Referencias

Repositorio del trabajo explicado:

https://github.com/santi-madrid/Arquitectura_de_Computadoras